Full NTT ASIC RTL handoff summary

Status

The project now contains a full 256-point NTT ASIC-compatible RTL handoff top.

Recommended handoff top:

- ‘ntt_core_256‘

This supersedes the earlier ‘ntt_core_8‘ prototype. The 8-point core remains in the repository as a bring-up/reference block, but it is not the final target.

Original FPGA/HLS context

The original project used Vitis/Vivado/HLS-style implementation files with FPGA-oriented constructs such as HLS pragmas, AXI interface inference, BRAM binding directives, and generated implementation artifacts.

Those files remain useful as algorithm/interface reference material, but the clean ASIC handoff path is the hand-written RTL under:

- ‘rtl/‘

The OpenLane owner should not use generated FPGA/HLS implementation files under ‘hls-design/‘ as the physical-design source unless deliberately comparing against the old implementation.

Clean ASIC-oriented RTL created

Arithmetic/datapath RTL:

- ‘rtl/mod_add.sv‘
- ‘rtl/mod_sub.sv‘
- ‘rtl/modmul_montgomery.sv‘
- ‘rtl/ntt_butterfly.sv‘

Full target RTL:

- ‘rtl/ntt_core_256.sv‘

Bring-up/reference RTL retained:

- ‘rtl/simple_dual_port_ram.sv‘
- ‘rtl/ntt_stage_2.sv‘
- ‘rtl/ntt_stage_4.sv‘
- ‘rtl/ntt_stage_8.sv‘
- ‘rtl/ntt_core_8.sv‘
- ‘rtl/ntt_stage_8_registered.sv‘

Use ‘ntt_core_256‘ for the full-design handoff.

What ‘ntt_core_256‘ implements

‘ntt_core_256‘ implements a fixed 256-point HLS-style NTT schedule with:

- ‘N = 256‘
- ‘Q = 8380417‘
- ‘Q_INV = 4236238847‘
- load interface for 256 input data words
- load interface for 256 bit-reversal table entries
- load interface for 255 twiddle values
- ‘start‘, ‘busy‘, and ‘done‘ control
- readback interface for 256 result words
- one shared butterfly datapath
- ping/pong internal storage for stage results
- 8 NTT stages
- 128 butterflies per stage

The controller performs:

1. Input permutation into ping storage using the bit-reversal table.
2. Stage iteration from length 1 through length 128.
3. HLS-style flattened butterfly index generation.
4. Twiddle lookup using the stage offset and butterfly-local index.
5. Butterfly execution through the verified modular multiplier/add/sub datapath.
6. Ping-pong writes between internal buffers.
7. Final result readback through ‘read_addr‘ and ‘read_data‘.

Verification completed

The following checks passed locally:

PASS: tb_ntt_core_256
PASS: ntt_core_256 Yosys synthesizability check completed
PASS: Full NTT OpenLane handoff readiness checks completed

The readiness script is:

- ‘scripts/check_openlane_handoff_readiness.sh‘

It runs:

- ‘scripts/run_ntt_core_256_tb.sh‘
- ‘scripts/lint_rtl.sh‘
- ‘scripts/run_synth_ntt_core_256.sh‘

Golden model and vectors

The full 256-point RTL testbench uses the Python golden model:

- ‘sim/ntt_golden.py‘

Generated vectors:

- ‘sim/test_vectors/input_256.hex‘
- ‘sim/test_vectors/bitrev_256.hex‘
- ‘sim/test_vectors/twiddles_256.hex‘
- ‘sim/test_vectors/expected_256.hex‘
- ‘sim/test_vectors/params_256.txt‘

The testbench is:

- ‘sim/tb_ntt_core_256.sv‘

Handoff support files

Simulation/test:

- ‘scripts/run_ntt_core_256_tb.sh‘

Lint/synthesis:

- ‘scripts/lint_rtl.sh‘
- ‘scripts/synth_ntt_core_256_yosys.tcl‘
- ‘scripts/run_synth_ntt_core_256.sh‘
- ‘build/synth_ntt_core_256_yosys.log‘

OpenLane starter files:

- ‘constraints/ntt_core_256.sdc‘
- ‘openlane/ntt_core_256/config.tcl‘
- ‘docs/openlane_handoff.md‘

SRAM/BRAM status

The clean handoff RTL does not instantiate known FPGA BRAM primitives such as:

- ‘RAMB18E1‘
- ‘RAMB36E1‘
- ‘xpm_memory_*‘
- Vivado block-memory IP

The full top currently uses inferred/register-array storage for functional RTL handoff and OpenLane bring-up. This is ASIC-compatible RTL, but a production tapeout flow should review the memory architecture and replace or wrap larger memories with process-specific SRAM macros once the target PDK and macro library are selected.

What remains before real tapeout

The project is ready for full-design OpenLane physical-flow exploration, but not automatic manufacturing signoff.

Remaining work for the physical-design owner:

1. Select exact PDK and standard-cell library.

2. Run technology-mapped synthesis.
3. Review timing, especially around the Montgomery multiplier and controller/memory paths.
4. Pipeline or restructure datapath if timing requires it.
5. Decide final SRAM macro strategy and replace/wrap inferred memories if needed.
6. Tune floorplan, utilization, IO constraints, and clock period.
7. Run placement, CTS, routing, DRC, LVS, antenna checks, extraction, STA, and signoff checks.
8. Generate and review final GDSII only after the above checks pass.

Correct handoff statement

Use this wording:

"Full 256-point NTT ASIC-compatible RTL and OpenLane starter handoff package."

Do not describe it as:

"Final tapeout-ready GDSII."